Aggrey — Role 2 (Data & Backend Lead) Learning Path

Project: Rackner AI Innovation Fellowship — Contract Obligation Extractor **Your job in one sentence:** turn a messy government contract PDF into clean, structured, source-cited obligations stored in a database that the rest of the team builds on. **Repo:** https://github.com/AggreyN/Rackner-Project

This guide is sequenced so you learn each skill **roughly the week before you need it**. Don't try to learn it all upfront — follow the calendar. Each section says *why you need it* and *how far to go* so you don't fall down rabbit holes.

How to use this document

- Skim the whole thing once so you know what's coming.
- Each week, open the matching section and watch/read **just enough** to do that week's deadline.
- Official docs are your real reference. Videos/courses are for getting unstuck fast.
- You don't need to master anything — you need to build the thing. Learn by doing.

---

0. Foundations (Week 1 — before anything else)

Why: you need a working environment before you can build. **Goal:** comfortable with the terminal, Git, Python virtual environments, and connecting to a database.

- The Missing Semester of Your CS Education (MIT) — shell, Git, the command line: https://missing.csail.mit.edu/
- Git & GitHub crash course — search "Git and GitHub for Beginners freeCodeCamp" on YouTube.
- Python virtual environments (official): https://docs.python.org/3/tutorial/venv.html

Stop when: you can clone the repo, make a branch, activate a venv, and run a Python file.

---

1. PDF Parsing & Ingestion (Weeks 2–3; OCR in Week 10)

Why: this is the front door of the whole product. Everything downstream depends on you reliably turning a PDF into text *that remembers what page and position it came from* (you need those coordinates later for click-to-highlight citations).

Primary tool — PyMuPDF (imported as `fitz`): - The Basics: https://pymupdf.readthedocs.io/en/latest/the-basics.html - Text extraction recipes: https://pymupdf.readthedocs.io/en/latest/recipes-text.html - GitHub (examples + searchable issues): https://github.com/pymupdf/pymupdf

Second tool — pdfplumber (great for tables + character-level positions): - GitHub + docs: https://github.com/jsvine/pdfplumber

For Week 10 (scanned/image PDFs that have no text layer): - Tables, scanned PDFs, and OCR walkthrough: https://www.nutrient.io/blog/extract-text-from-pdf-pymupdf/ - Tesseract OCR via `pytesseract` — search "pytesseract pdf2image tutorial".

Stop when: you can extract text page-by-page from a real SAM.gov solicitation and keep each block's page number.

---

2. SQL & PostgreSQL (Weeks 4–6)

Why: you own the database — the single source of truth the whole team writes to and reads from.

Learn SQL fundamentals first, then Postgres specifically: - freeCodeCamp — search **"Learn PostgreSQL – Full Course for Beginners"** on YouTube (~4 hrs, the standard starting point). - Hussein Nasser's PostgreSQL course (free, deeper on how the database actually works): https://www.classcentral.com/course/youtube-postgressql-93078 - Stephen Grider — *SQL and PostgreSQL: The Complete Developer's Guide* (Udemy, ~$15 on sale, very practical): https://www.udemy.com/course/sql-and-postgresql/ - Official Postgres learning hub (reference): https://www.postgresql.org/docs/online-resources/

Focus on: CREATE TABLE, primary/foreign keys, INSERT/SELECT/UPDATE, JOINs, indexes, and basic schema design. You don't need advanced query tuning yet.

Stop when: you can design and create the `documents`, `clauses`, `obligations`, `reviews`, `deadlines` tables and query across them.

---

3. Python ↔ Postgres, Models & Migrations (Weeks 4–5)

Why: your pipeline (Python) has to read/write the database, and your schema will change — migrations keep that sane and version-controlled.

- SQLAlchemy (the ORM — define tables as Python classes): https://docs.sqlalchemy.org/
- Alembic (database migrations — version control for your schema, required in Week 4): https://alembic.sqlalchemy.org/
- `psycopg` (the Postgres driver under the hood): https://www.psycopg.org/psycopg3/docs/

Stop when: you can define a table in Python, run a migration to create it, and insert/query rows.

---

4. Regex for Clause Segmentation (Week 3)

Why: FAR/DFARS clauses follow predictable numbering (52.xxx, 252.xxx). Regex is how you chop a contract into clean clause chunks before the AI reads them.

- Python regex HOWTO (official): https://docs.python.org/3/howto/regex.html
- regex101 — build and test your patterns live, with explanations: https://regex101.com/

Stop when: you can split a contract's text into a list of clauses, each tagged with its clause ID and page/offset.

---

5. FastAPI — Serving the Data (Week 8)

Why: Role 3's React app needs endpoints to upload documents, fetch obligations, and update review status. FastAPI is how you expose your work.

- FastAPI tutorial (one of the best docs anywhere — learn straight from it): https://fastapi.tiangolo.com/tutorial/
- Pydantic (data validation, used heavily by FastAPI): https://docs.pydantic.dev/

Stop when: you can run a local API with endpoints that read/write your Postgres tables, visible at `/docs`.

---

6. Your Data Source — SAM.gov (good to know; Role 3 owns integration)

Why: your demo PDFs are public federal solicitations from SAM.gov. No sensitive data, no CUI wall — that's the team's big practical advantage.

- SAM.gov "Get Opportunities" public API: https://open.gsa.gov/api/get-opportunities-public-api/
- Browse opportunities (to grab sample PDFs by hand for early testing): https://sam.gov/

Weekly map (matches your calendar deadlines)

Week (Fri due)	Deadline	Study this section
Wk1 · Jul 3	Dev env + repo + Postgres setup	0. Foundations
Wk2 · Jul 10	PDF ingestion v0	1. PDF Parsing
Wk3 · Jul 17	FAR/DFARS segmentation	4. Regex
Wk4 · Jul 24	Postgres schema + migrations	2. SQL/Postgres + 3. Models/Migrations
Wk5 · Jul 31	Ingestion → DB pipeline	3. Models/Migrations
Wk6 · Aug 7	Obligation register + tracking	2. SQL (queries)
Wk7 · Aug 14	Citation grounding + verification	1. PDF (coordinates)
Wk8 · Aug 21	FastAPI endpoints	5. FastAPI
Wk9 · Aug 28	Eval set storage + precision/recall	2. SQL + Python
Wk10 · Sep 4	Deadlines, CSV export, OCR fallback	1. PDF (OCR)
Wk11 · Sep 11	Integration freeze	— (polish)
Wk12 · Sep 18	**Demo Day**	— (you've got this)

YC startup videos (watch a couple early)

These two map directly onto what your team must prove this summer:

- **"How to Get and Evaluate Startup Ideas" – Jared Friedman** — keeps you on the wedge, out of scope creep. (YC YouTube / Library)
- **"How to Talk to Users" – Eric Migicovsky** — your customer-discovery playbook for the eval-set interviews.

More: - *How to Start a Startup* (Sam Altman / Stanford, 20 lectures): https://www.youtube.com/playlist?list=PL5q_lef6zVkaTY_cT1k7qFNF2TidHCe-1 — start: https://www.youtube.com/watch?v=CBYhVcO4WgI - *The Best Way to Launch Your Startup*: https://www.youtube.com/watch?v=u36A-YTxiOw - YC Startup School (free curriculum): https://www.startupschool.org/curriculum - YC Library: https://www.ycombinator.com/library - YC YouTube channel (search the talks above): https://www.youtube.com/@ycombinator

---

Mantra for the summer: learn just-in-time, build in small slices, and keep everything traceable to the source. You're the backbone of this team — when your pipeline is solid, everyone else moves fast.